



Conception et Programmation Orientées Objet

MP1-DS

Akil ELKAMEL

akil.elkamel@isimm.u-monastir.tn

Support de cours, slides, TPs etc...



<https://github.com/akilelkamel/CP00/>

Plan du cours

- Introduction aux systèmes d'information et au développement logiciel
- Cycle de vie du développement logiciel
- Analyse et modélisation orientées objet avec UML
- Programmation Orientée Objet (POO)
 - Les bases du langage Java
 - La POO avec Java (Classe, objets, constructeur, instanciation ...)
 - Des concepts avancés de la POO avec Java (Heritage, polymorphisme, classes abstraites, interfaces)

Chapitre 1:

Introduction

Les systèmes d'information

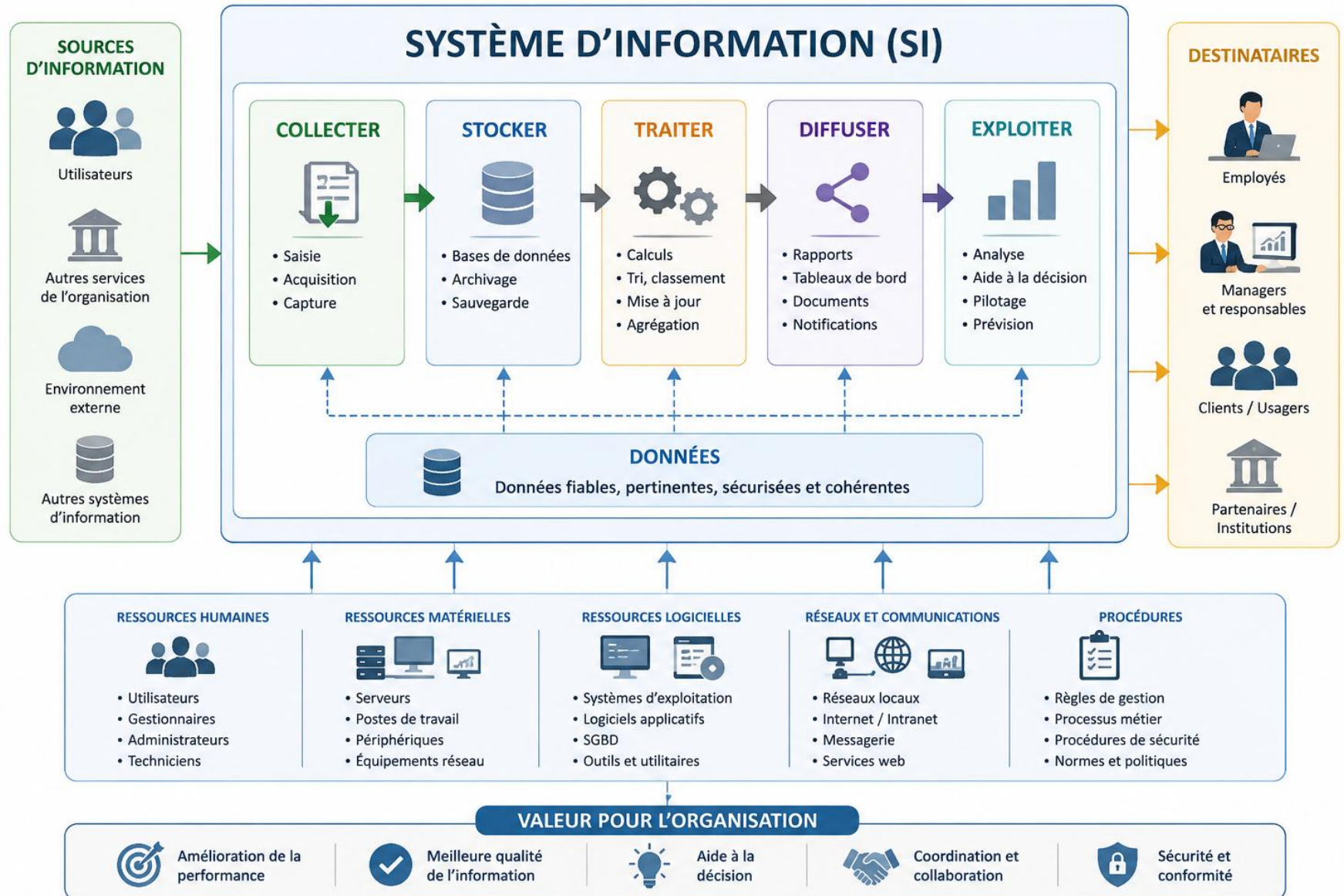
Le développement logiciel

1.1. Introduction aux systèmes d'information

Systeme d'information

- Un système d'information (SI) est un ensemble organisé de ressources humaines, matérielles, logicielles, données et procédures permettant de collecter, stocker, traiter, diffuser et exploiter l'information au sein d'une organisation afin de soutenir ses activités et sa prise de décision.

Système d'information



Les principales composantes d'un SI

Un système d'information s'appuie généralement sur:

- **Personnes:** utilisateurs, gestionnaires, administrateurs...
- **Données:** informations manipulées par l'organisation.
- **Logiciels:** applications et systèmes permettant de traiter les données.
- **Matériels:** ordinateurs, serveurs, périphériques, équipements réseau...
- **Procédures:** règles et processus définissant comment l'information est gérée.
- **Réseaux:** moyens permettant la communication et l'échange des informations.

Les rôles d'un système d'information

- Collecter l'information
 - Acquérir les données provenant des utilisateurs, des systèmes ou de l'environnement.
- Stocker l'information
 - Conserver les données de manière organisée et accessible.
- Traiter l'information
 - Transformer les données brutes en informations exploitables: calcul, classement, filtrage, agrégation, etc.
- Diffuser et communiquer l'information
 - Mettre l'information à disposition des personnes ou systèmes qui en ont besoin.

Les rôles d'un système d'information

- Faciliter la prise de décision
 - Fournir des informations pertinentes permettant aux responsables de prendre des décisions.
- Coordonner les activités
 - Faciliter la circulation de l'information entre les différents services et acteurs de l'organisation.
- Automatiser les processus
 - Prendre en charge automatiquement certaines tâches répétitives ou règles de gestion.
- Assurer la traçabilité
 - Conserver l'historique des opérations et des événements afin de pouvoir suivre les actions réalisées.

Les principales étapes de réalisation d'un SI

ÉTAPES DE RÉALISATION D'UN SYSTÈME D'INFORMATION (SI)



ACTEURS IMPLIQUÉS TOUT AU LONG DU PROJET



Utilisateurs



Analystes



Concepteurs



Développeurs



Testeurs



Administrateurs



Chefs de projet

1. Analyse des besoins

- **Objectif:** comprendre ce que le système doit faire.
- On cherche à identifier:
 - les besoins des utilisateurs
 - les fonctionnalités attendues
 - les acteurs du système
 - les contraintes
 - les règles métier
 - les exigences non fonctionnelles: sécurité, performance, disponibilité, etc.
- **Question principale:** Que doit faire le système ?
- **Exemple:** pour une bibliothèque:
 - gérer les livres
 - gérer les retours
 - gérer les adhérents
 - produire des statistiques
 - enregistrer les emprunts

2. Analyse du système

- **Objectif:** comprendre et structurer le problème.
- On identifie:
 - les principales entités du domaine
 - leurs caractéristiques
 - leurs responsabilités
 - les interactions
 - les processus métier
 - les relations entre les différents éléments.
- C'est à ce niveau que l'on commence à construire une *représentation abstraite du système*.
- **Question principale:** Quels sont les éléments du problème et comment interagissent-ils ?

3. Conception

- **Objectif:** définir comment le système sera construit.
- On détermine notamment:
 - l'architecture du système
 - les composants/modules
 - les classes et objets
 - les relations entre classes
 - les interfaces
 - les bases de données
 - les mécanismes de communication.
- Pour une approche orientée objet, on peut utiliser **UML** pour représenter la conception:
 - diagramme de classes
 - diagramme de séquence
 - diagramme d'activités
 - etc.
- **Question principale:** Comment organiser le système pour répondre aux besoins ?

4. Implémentation

- **Objectif:** transformer la conception en logiciel fonctionnel.
- Cette étape comprend:
 - écriture du code
 - création des classes
 - implémentation des méthodes
 - création des bases de données
 - intégration des composants
 - utilisation des bibliothèques et frameworks.
- **Question principale:** Comment transformer la conception en un programme ?

5. Tests et validation

- **Objectif:** vérifier que le système fonctionne correctement et répond aux besoins.
- On peut réaliser:
 - tests unitaires
 - tests d'intégration
 - tests système
 - tests fonctionnels
 - tests de performance
 - tests de sécurité
 - tests d'acceptation.
- **Question principale:** Le système fait-il ce qui était demandé ?

6. Déploiement

- **Objectif:** rendre le système disponible pour ses utilisateurs.
- Cela peut comprendre:
 - installation
 - configuration des serveurs
 - configuration des bases de données
 - migration des données
 - configuration réseau
 - formation des utilisateurs
 - mise en production.
- **Question principale:** Comment rendre le logiciel disponible dans un environnement réel d'utilisation?

7. Maintenance et évolution

- **Objectif:** maintenir le système opérationnel et le faire évoluer.
- Après son déploiement, le système peut nécessiter :
 - correction des erreurs
 - amélioration des performances
 - ajout de fonctionnalités
 - adaptation à de nouvelles contraintes
 - mise à jour des technologies
 - amélioration de la sécurité
- C'est une étape particulièrement importante en conception orientée objet, car une bonne conception doit faciliter l'évolution du système.
- **Question principale:** Comment maintenir le système opérationnel et l'adapter aux nouveaux besoins ?

Étapes de réalisation d'un système d'information

Etape	Question principale	Résultat
1. Analyse des besoins	Que veut l'utilisateur ?	Cahier des besoins
2. Analyse	Que contient le problème ?	Modèle du domaine
3. Conception	Comment construire le système ?	Modèle de conception
4. Implémentation	Comment programmer la solution ?	Code source
5. Tests	Le système fonctionne-t-il correctement ?	Système validé
6. Déploiement	Comment mettre le système en production ?	Système opérationnel
7. Maintenance	Comment le maintenir et le faire évoluer ?	Versions évolutives

1.2. Introduction au développement logiciel

Qu'est-ce qu'un logiciel ?

- Un logiciel (software) est un ensemble organisé de programmes, de données et de ressources permettant à un système informatique d'accomplir certaines tâches.
- Un logiciel peut comprendre:
 - des programmes / code source
 - des données
 - des fichiers de configuration
 - des bibliothèques
 - des interfaces utilisateur
 - de la documentation
- Exemples :
 - Système d'exploitation: Windows, Linux
 - Applications: navigateur Web, traitement de texte, IDE
 - Applications mobiles
 - Systèmes embarqués
 - Systèmes d'information d'entreprise

Transformer un problème réel en logiciel

- Un logiciel est généralement développé pour **résoudre un problème réel ou répondre à un besoin**.



- **Exemple : système de gestion universitaire**
 - Besoin: Permettre de gérer les étudiants, leurs inscriptions, gérer les enseignants et les notes, etc...
 - Ce besoin doit être transformé progressivement en:
 - fonctionnalités
 - modèles
 - architecture
 - classes et objets
 - code
 - tests.
- **Peut-on commencer directement par écrire le code ?**

Pourquoi la conception est-elle nécessaire ?

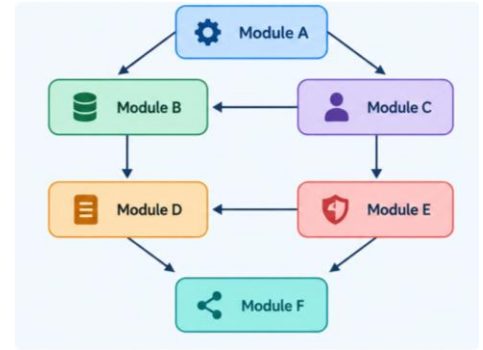
- Pour un petit programme: Problème ➔ Code ➔ Résultat
- Pour un système complexe :
 - Interface utilisateur
 - Logique métier
 - Gestion des données
 - Sécurité
 - Services externes
- Plus le système devient complexe, plus il devient nécessaire de :
 - structurer le logiciel
 - identifier les responsabilités
 - organiser les composants
 - gérer les dépendances
 - faciliter les modifications
 - assurer la qualité du logiciel.

La complexité des logiciels

- La complexité augmente avec :
 - le nombre de fonctionnalités
 - le volume de données
 - le nombre d'utilisateurs
 - les interactions entre composants
 - les contraintes de sécurité
 - les contraintes de performance
 - l'intégration avec d'autres systèmes
 - l'évolution des besoins.
- **Exemple:**
 - Une application de commerce électronique peut intégrer des clients, Produits, Commandes, Paiements, Livraison, Fournisseurs
 - Chaque élément interagit avec plusieurs autres.

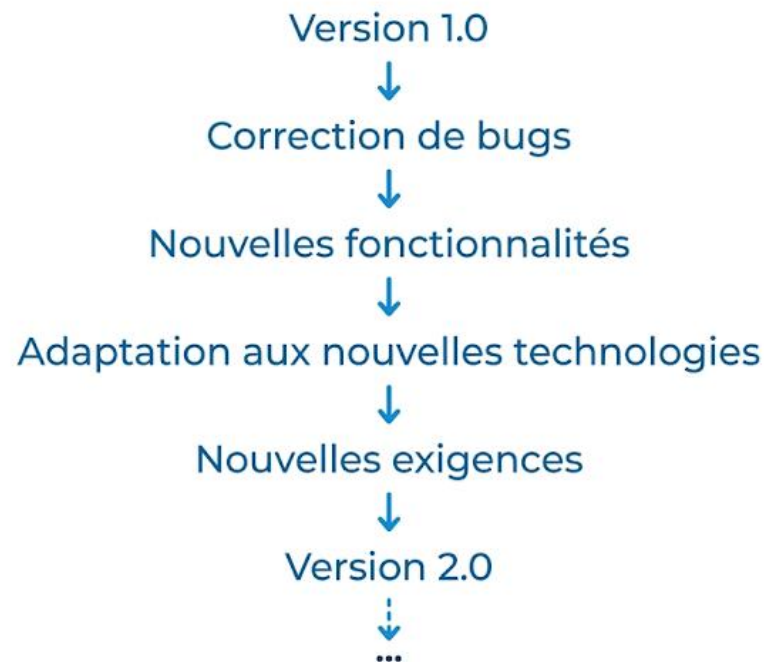
Quand le logiciel devient difficile à maintenir

- Considérons un système contenant de nombreuses fonctionnalités:
- Une modification dans un module peut avoir des conséquences sur plusieurs autres.
- Cela peut conduire à:
 - fort couplage
 - duplication du code
 - difficultés de compréhension
 - difficultés de test
 - effets de bord
 - augmentation du coût de maintenance.
- Plus un logiciel est complexe, plus son organisation devient importante.



Le logiciel évolue (maintenance)

- Un logiciel n'est généralement pas terminé après sa première version.
- Après son déploiement, il doit évoluer :



Le logiciel évolue (maintenance)

- Types de maintenance :

Type	Objectif
Corrective	Corriger les erreurs
Évolutive	Ajouter/modifier des fonctionnalités
Adaptative	Adapter le logiciel à son environnement
Préventive	Améliorer la maintenabilité et éviter des problèmes futurs

Un bon logiciel doit être conçu non seulement pour fonctionner aujourd'hui, mais également pour **pouvoir évoluer demain.**

Evolution du développement logiciel

- Avec l'évolution :
 - les logiciels deviennent plus grands
 - les équipes deviennent plus importantes
 - les besoins deviennent plus complexes
 - les systèmes doivent évoluer plus rapidement.
- Conséquence :
 - Il ne suffit plus de savoir programmer. Il faut également savoir concevoir.

Programmer et concevoir: deux activités différentes

- **Programmer:**
 - Traduire une solution en instructions exécutables.
- **Concevoir :**
 - Déterminer comment organiser les différentes parties du logiciel pour résoudre le problème.
- Une bonne conception cherche notamment à obtenir:
 - une organisation claire
 - des responsabilités bien définies
 - un faible couplage
 - une forte cohésion
 - une bonne réutilisabilité
 - une bonne maintenabilité

Que doit-on décider lors de la conception ?

- Lorsqu'on conçoit un logiciel, on doit répondre à plusieurs questions :
 - **Quels sont les éléments du système ?**
Classes / objets / composants
 - **Quelles responsabilités possèdent-ils ?**
Attributs / méthodes / services
 - **Comment interagissent-ils ?**
Associations / messages / appels
 - **Comment organiser le système ?**
Architecture / modules / packages
 - **Comment le système pourra-t-il évoluer ?**
Abstraction / encapsulation / extensibilité